🔌 Redes y Sockets

La base de las comunicaciones en Java

Desde la consola al mundo real: TCP e I/O Streams

Empezar ➡️

Hoja de Ruta

Lo dividiremos en **3 Grandes Fases** que van de la teoría pura a un servidor concurrente real:

1

Teoría

Arquitectura Cliente-Servidor, el Socket como Endpoint y los Streams I/O en Java.

2

Práctica

Del *Ejemplo 0* de sintaxis pura hasta Request/Response y sesiones persistentes con bucle `while`.

3

Caso de uso real

Servidor multi-cliente con Hilos: **Chat en tiempo real** entre múltiples usuarios simultáneos.

1.1 La Arquitectura Cliente-Servidor 🏢

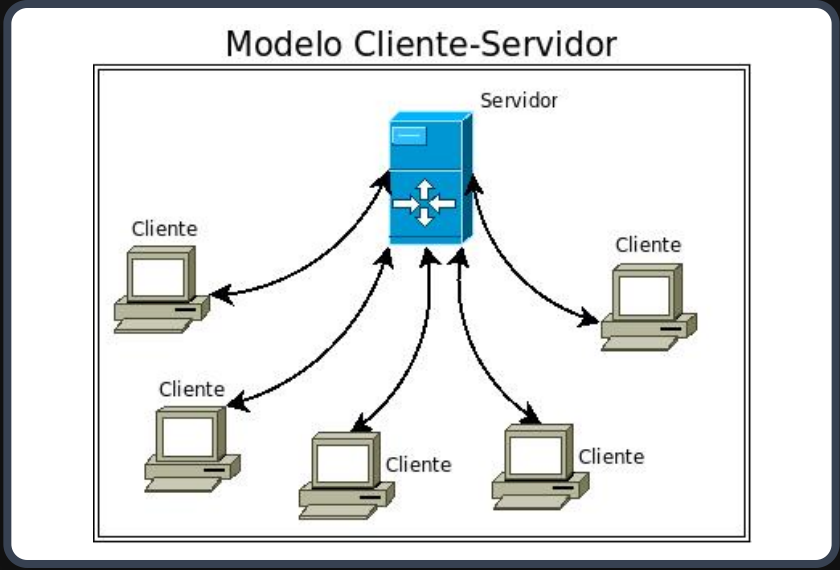
Cualquier sistema conectado del mundo se apoya en que alguien **Pide (Cliente)** y alguien **Escucha y Responde (Servidor)**.

🧑‍💻 El Cliente (Iniciador)

- Siempre **inicia la conexión** hacia la IP + Puerto del servidor.
- Duración corta en red (conecta → pide → recibe → cierra).
- Ej: Tu navegador, una app de móvil, un cliente de chat.

🖥️ El Servidor (Oyente)

- Programa de **bucle infinito** esperando llamadas en un Puerto fijo.
- Procesa peticiones y devuelve respuestas a múltiples clientes.
- Ej: Google.com, un servidor de chat, una API REST en producción.



1.2 ¿Qué es un Socket? Visualmente 🔌

Un Socket es el "enchufe" virtual que el Sistema Operativo crea para conectar dos procesos por red mediante IP + Puerto.

Secuencia de conexión TCP

En otros lenguajes (misma idea)

```python
# Python 3
import socket # Importamos la librería de red
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Instanciamos el socket TCP
s.connect(("localhost", 8080)) # Nos conectamos a la IP y Puerto del servidor
```

```node
// Node.js
const net = require('net'); // Importamos librería nativa de red
const client = net.createConnection({ port: 8080 }); // Abre la conexión al proceso escuchando
```

TCP vs UDP:

🛡️ **TCP:** Orientado a conexión. Garantiza orden y entrega íntegra. *(Lo que usamos)*

1.3 Ejemplos Completos Multilenguaje 🌐

Para demostrar que los sockets son universales, veamos un ejemplo básico y completo de un cliente TCP en lenguaje Python y Node.js.

🐍 Cliente Completo Python

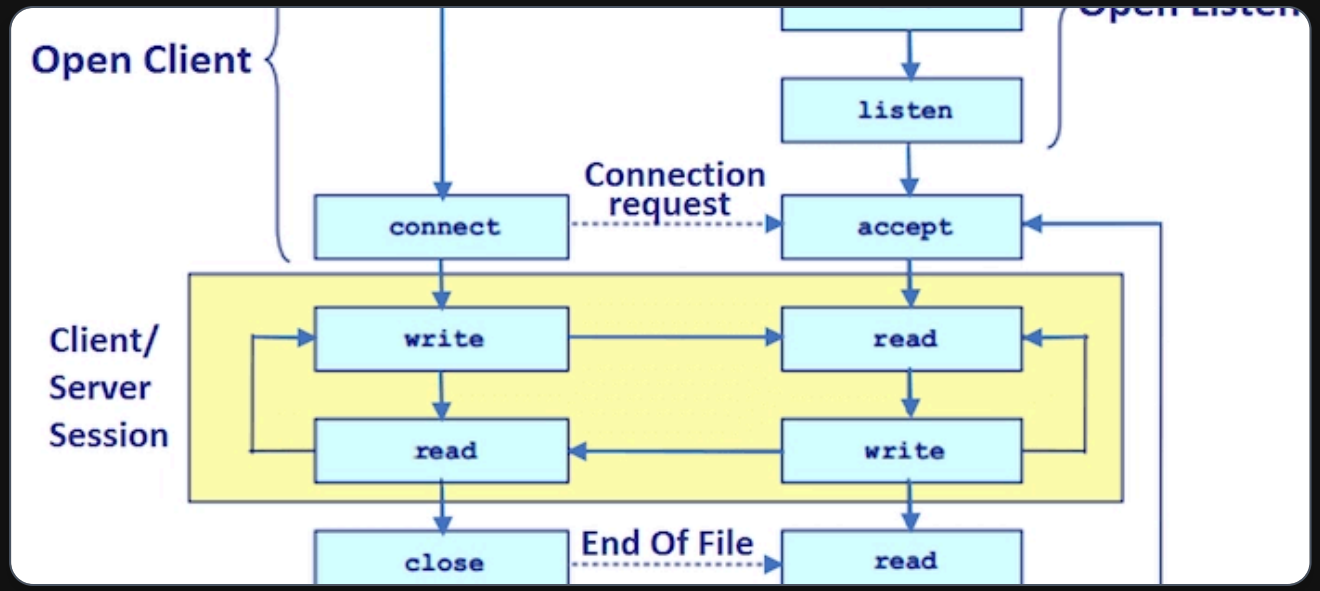
```python
import socket

# 1. Crear socket TCP
cliente = socket.socket(socket.AF_INET, socket.SOC
# 2. Conectar al servidor
cliente.connect(('localhost', 8080))

# 3. Enviar mensaje convertido a bytes
cliente.sendall(b'Hola Servidor Python\n')
# 4. Recibir respuesta leyendo max 1024 bytes
data = cliente.recv(1024)
print(f"Recibido: {data.decode('utf-8')}")

cliente.close() # 5. Cerrar
```

🟩 Cliente Completo Node.js

```node
const net = require('net');

// 1. Crear cliente y conectar
const cliente = net.createConnection({ port: 8080
  // 2. Al conectar existosamente, enviar informac
  cliente.write('Hola Servidor Node!\n');
});

// 3. Registrar evento asíncrono para leer datos r
cliente.on('data', (data) => {
  console.log(`Recibido: ${data.toString()}`);
  cliente.end(); // 4. Cerrar la desconexión
});
```

2. Introducción a Java Sockets ☕

En Java, usaremos **Sockets TCP** en un 99% de los casos. La API del paquete `java.net` facilita enormemente el manejo escondiendo la complejidad real del sistema.



2.1 Protocolos: TCP vs UDP

DIFERENCIA CLAVE PARA EL ALUMNO

Tipo	Clase Java	Conexión	Fiabilidad
TCP	`Socket / ServerSocket`	Sí (Tubería)	Alta
UDP	`DatagramSocket`	No (Paquetes)	Baja

Mientras TCP asegura íntegramente la entrega en orden (ideal para BBDD o Chats), UDP lanza los datos al vacío sin esperar confirmación (útil para Videojuegos online o Streaming).

2.2 Funcionamiento del Servidor Java

Para establecer nuestra comunicación fiable TCP, usamos dos clases muy marcadas en el ecosistema Java que encapsulan toda esa complejidad de la red.

1 java.net.ServerSocket

Es la "operadora". Se asigna a un puerto del SO con `new ServerSocket(PORT)`.

Llamar a su método `.accept()` hará que el programa **bloquee la ejecución** (quede congelado) hasta que llegue y se conecte un cliente físico.

2 java.net.Socket

Es el tubo de la conexión activa. Lo devuelve la operadora al hacer `accept()`.

Tiene sus propios *InputStream* y *OutputStream* acoplados para recibir y enviar los textos o datos de una forma natural.

2.3 Las Dos Clases Maestras de Java 🧬

En `java.net` Java distingue fuertemente el rol Cliente del rol Servidor en dos clases separadas (basado en el modelo de Sockets bidireccionales):

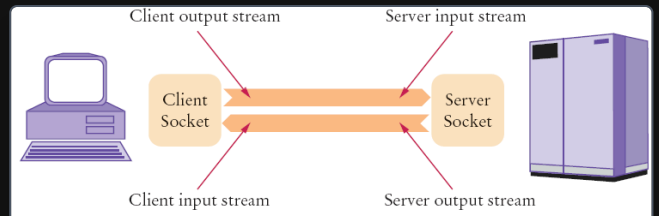


Figure 5 Client and Server Sockets

🌍 java.net.Socket

El "Cable Activo". El cliente lo crea para conectarse. El servidor lo recibe al hacer `accept()`.

```
// Cliente: conectar al servidor
Socket socket = new Socket("127.0.0.1", 9090)
```

🎧 java.net.ServerSocket

La "Operadora". Se ancla a un puerto del OS y **bloquea el programa** hasta que llega una conexión.

2.4 Flujo de Datos: I/O Streams 🌊

Un Socket funciona igual que leer/escribir en un fichero. Necesitamos envolver los bytes en clases de texto cómodas:

📤 Para Enviar (OutputStream)

Envolvemos el Stream de bytes con `PrintWriter` para usar `.println()` directamente.

```java
PrintWriter out = new PrintWriter(
    socket.getOutputStream(), true // auto-flush
);
out.println("Mensaje de texto por red");
```

📥 Para Leer (InputStream)

⚠️ BLOQUEO: `readLine()` congela el hilo hasta recibir datos de la red.

```java
BufferedReader in = new BufferedReader(
    new InputStreamReader(socket.getInputStream())
);
String texto = in.readLine(); // ← hilo bloqueado aquí
```

Ejemplo 0: Sintaxis Pura — Hola TCP

El esqueleto mínimo. Sin contexto, solo para ver la estructura

SERVIDOR Abre puerto, espera y lee

```java
import java.io.*;
import java.net.*;

public class Servidor {
    public static void main(String[] args) throws Exc
        // 1. Abrir puerto 1234 y esperar conexión
        ServerSocket server = new ServerSocket(1234);
        System.out.println("Esperando...");

        Socket cliente = server.accept(); // ← BLOQUE
        System.out.println("¡Cliente conectado!");

        // 2. Leer mensaje del cliente
        BufferedReader in = new BufferedReader(
            new InputStreamReader(cliente.getInputStr
        );
        System.out.println("Recibido: " + in.readLine

        cliente.close();
        server.close();
```

CLIENTE Conecta y envía

```java
import java.io.*;
import java.net.*;

public class Cliente {
    public static void main(String[] args) throws Exc
        // 1. Conectarse al servidor
        Socket socket = new Socket("localhost", 1234)
        System.out.println("Conectado al servidor.");

        // 2. Enviar un mensaje
        PrintWriter out = new PrintWriter(
            socket.getOutputStream(), true
        );
        out.println("Hola desde el cliente!");

        socket.close();
    }
}
```

3.1 Ejemplo 1: Bidireccional — Calculadora Remota

Escenario: El cliente envía una operación (`SUMA:5:3`) y el servidor la calcula y devuelve el resultado.

CLIENTE

```
1   import java.io.*; import java.net.*;
2
3   public class ClienteCalc {
4     public static void main(String[] args) throws Ex
5       try (Socket s = new Socket("localhost", 7000);
6            PrintWriter out = new PrintWriter(s.getOu
7            BufferedReader in = new BufferedReader(
8                new InputStreamReader(s.getInputStrea
9
10          // Enviar operación al servidor remoto
11          out.println("SUMA:5:3");
12
13          // Leer resultado calculado
14          String resultado = in.readLine();
15          System.out.println("Resultado: " + resulta
16      }
```

SERVIDOR

```
1   import java.io.*; import java.net.*;
2
3   public class ServidorCalc {
4     public static void main(String[] args) throws Ex
5       try (ServerSocket ss = new ServerSocket(7000);
6            Socket c = ss.accept();
7            PrintWriter out = new PrintWriter(c.getOu
8            BufferedReader in = new BufferedReader(
9                new InputStreamReader(c.getInputStrea
10
11          String peticion = in.readLine(); // "SUMA:
12          String[] partes = peticion.split(":");
13          int resultado = switch (partes[0]) {
14              case "SUMA" -> Integer.parseInt(partes
15              case "RESTA" -> Integer.parseInt(parte
16              default -> -1;
```

3.2 Ejemplo 2: Sesión Persistente — Calculadora Interactiva

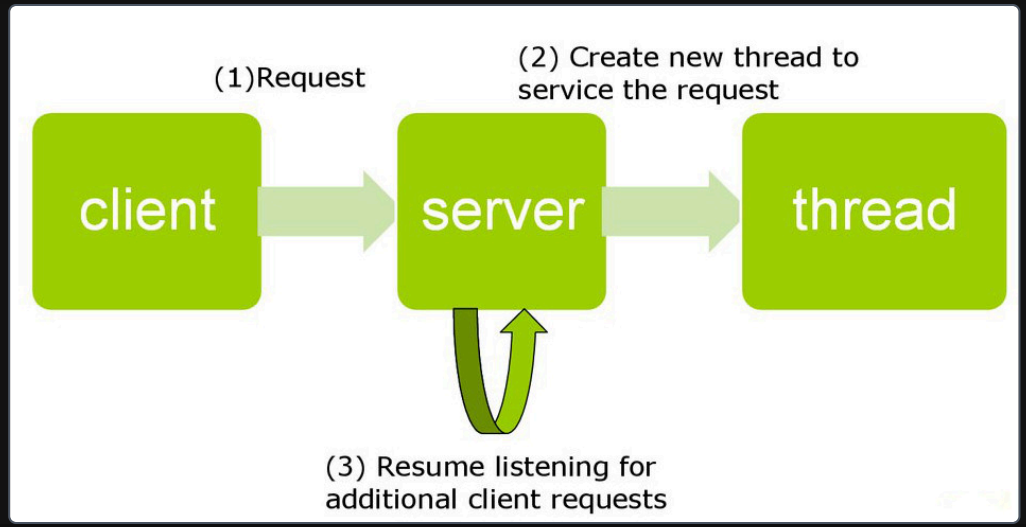
Problema: El ejemplo anterior muere tras 1 operación. Añadimos bucle `while` para mantener la sesión abierta.

```java
import java.io.*; import java.net.*;

public class ServidorCalcInteractivo {
  public static void main(String[] args) throws Exception {
    try (ServerSocket ss = new ServerSocket(7001);
         Socket c = ss.accept();
         PrintWriter out = new PrintWriter(c.getOutputStream(), true);
         BufferedReader in = new BufferedReader(new InputStreamReader(c.getInputStream()))) {

        out.println("Calculadora Remota 🧮. Escribe SUMA:a:b | RESTA:a:b | EXIT");
        String linea;

        // El servidor lee en bucle hasta que el cliente mande EXIT o cierre
        while ((linea = in.readLine()) != null) {
            if (linea.equalsIgnoreCase("EXIT")) { out.println("Adiós!"); break; }

            String[] p = linea.split(":");
            try {
```

🚀 CONCURRENCIA

Ejemplo 3: Chat Multi-Usuario con Hilos

Del servidor de 1 cliente al servidor de ∞ clientes simultáneos.



3.3 Chat en Tiempo Real — Servidor Multihilo ⚡

Escenario: Múltiples alumnos conectan al mismo servidor de chat. Cada uno en su propio hilo, los mensajes se retransmiten a todos (broadcast).

ServidorChat.java

```
1   import java.io.*; import java.net.*;
2   import java.util.*;
3
4   public class ServidorChat {
5     // Lista compartida de todos los clientes
6     static List<PrintWriter> clientes = new ArrayL
7
8     public static void main(String[] args) throws
9       ServerSocket server = new ServerSocket(9000)
10      System.out.println("Chat Server en :9000");
11
12      while (true) {
13        Socket nuevo = server.accept();
14        // Instanciamos nuestro Thread personaliza
15        HiloCliente hilo = new HiloCliente(nuevo);
16        hilo.start();
17      }
```

ClienteChat.java

```
1   import java.io.*; import java.net.*;
2   import java.util.Scanner;
3
4   public class ClienteChat {
5     public static void main(String[] args) throws
6       Socket s = new Socket("localhost", 9000);
7       PrintWriter out = new PrintWriter(s.getOutpu
8       BufferedReader in = new BufferedReader(new I
9       Scanner teclado = new Scanner(System.in);
10
11      System.out.print("Tu apodo: ");
12      out.println(teclado.nextLine()); // Enviar e
13
14      // Hilo separado (Implementando Runnable)
15      class HiloLector implements Runnable {
16        public void run() {
17          try {
```

Buenas Prácticas: No Pierdas Recursos

⚠️ Fugas de Recursos (Memory Leak)

Un `Socket` abierto consume recursos a nivel del **Kernel del SO**. Si no cierras con `.close()`, acumulas miles de conexiones zombi hasta que el sistema colapsa.

Solución: Usa siempre `try-with-resources`. La JVM cierra automáticamente aunque haya excepción.

⏱️ Bloqueos en el Hilo Principal (UI Freeze)

`readLine()` y `accept()` son **bloqueantes**. Si los metes en el hilo de la UI (Swing, JavaFX, Android), la pantalla se congela.

Solución: Delega siempre la E/S de red a un `Thread` secundario o usa `socket.setSoTimeout(5000)` para no esperar eternamente.

□ Las 3 Excepciones Míticas de Red

Inevitables en tus prácticas. Apréndetelas antes de chocar con ellas:

`BindException`	**Address already in use.** Has lanzado tu `Servidor.java` dos veces y el puerto OS sigue ocupado con el proceso anterior. Matar el proceso en el IDE o cambiar el número de puerto.
`ConnectException`	**Connection refused.** El cliente intentó llamar pero nadie responde en ese puerto. Verifica que el servidor está corriendo antes de lanzar el cliente.
`SocketException`	**Connection reset.** Estabas leyendo tranquilamente y el otro extremo cerró el socket abruptamente (red caída, proceso matado). Tu bucle `while` recibirá `null` en `readLine()`. Controla ese `null`.

🔮 El Siguiente Nivel: de Sockets a Spring Boot

La API `java.net` es la base real de internet. **Spring Boot la abstrae completamente, pero sin entenderla, no podrás debugar lo que pasa por debajo.**

🔌 Sockets TCP

- Control total, complejidad total.
- Tú defines el protocolo (`"SUMA:5:3"`), el parsing y el multihilo.
- Útil en IoT, videojuegos, drivers de bajo nivel.

🍃 Spring Boot (Próximamente)

- Tomcat gestiona el `ServerSocket` por ti en background.
- Una petición HTTP `GET` simplemente abre un Socket TCP, manda el request, recibe la respuesta y cierra.
- **WebSocket en Spring:** El equivalente a nuestro Chat Multihilo, pero sobre protocolo web estándar.